
Strictly Batch Imitation Learning by Energy-based Distribution Matching

Daniel Jarrett^{1*}, Ioana Bica^{24*}, Mihaela van der Schaar¹³⁴

¹University of Cambridge, UK, ²University of Oxford, UK,
³University of California, Los Angeles, ⁴Alan Turing Institute, UK
daniel.jarrett@maths.cam.ac.uk, ioana.bica@eng.ox.ac.uk

Abstract

Consider learning a policy purely on the basis of demonstrated behavior—that is, with no access to reinforcement signals, no knowledge of transition dynamics, and no further interaction with the environment. This *strictly batch imitation learning* problem arises wherever live experimentation is costly, such as in healthcare. One solution is simply to retrofit existing algorithms for apprenticeship learning to work in the offline setting. But such an approach leans heavily on off-policy evaluation or offline model estimation, and can be indirect and inefficient. We argue that a good solution should be able to explicitly parameterize a policy (i.e. respecting action conditionals), implicitly learn from rollout dynamics (i.e. leveraging state marginals), and—crucially—operate in an entirely offline fashion. To address this challenge, we propose a novel technique by *energy-based distribution matching* (EDM): By identifying parameterizations of the (discriminative) model of a policy with the (generative) energy function for state distributions, EDM yields a simple but effective solution that equivalently minimizes a divergence between the occupancy measure for the demonstrator and a model thereof for the imitator. Through experiments with application to control and healthcare settings, we illustrate consistent performance gains over existing algorithms for strictly batch imitation learning.

1 Introduction

Imitation learning deals with training an agent to mimic the actions of a demonstrator. In this paper, we are interested in the specific setting of *strictly batch imitation learning*—that is, of learning a policy purely on the basis of demonstrated behavior, with no access to reinforcement signals, no knowledge of transition dynamics, and—importantly—no further interaction with the environment. This problem arises wherever live experimentation is costly, such as in medicine, healthcare, and industrial processes. While behavioral cloning is indeed an intrinsically offline solution as such, it fails to exploit precious information contained in the distribution of states visited by the demonstrator.

Of course, given the rich body of recent work on (online) apprenticeship learning, one solution is simply to repurpose such existing algorithms—including classic inverse reinforcement learning and more recent adversarial imitation learning methods—to operate in the offline setting. However, this strategy leans heavily on off-policy evaluation (which is its own challenge per se) or offline model estimation (inadvisable beyond small or discrete models), and can be indirect and inefficient—via off-policy alternating optimizations, or by running RL in a costly inner loop. Instead, we argue that a good solution should directly parameterize a policy (i.e. respect action conditionals), account for rollout dynamics (i.e. respect state marginals), and—crucially—operate in an entirely offline fashion without recourse to off-policy evaluation for retrofitting existing (but intrinsically online) methods.

Contributions In the sequel, we first formalize imitation learning in the *strictly batch* setting, and motivate the unique desiderata expected of a good solution (Section 2). To meet this challenge, we propose a novel technique by *energy-based distribution matching* (EDM) that identifies parameterizations

* Authors contributed equally

of the (discriminative) model of a policy with the (generative) energy function for state distributions (Section 3). To understand its relative simplicity and effectiveness for batch learning, we relate the EDM objective to existing notions of divergence minimization, multitask learning, and classical imitation learning (Section 4). Lastly, through experiments with application to control tasks and healthcare, we illustrate consistent improvement over existing algorithms for offline imitation (Section 5).

2 Strictly Batch Imitation Learning

Preliminaries We work in the standard Markov decision process (MDP) setting, with states $s \in \mathcal{S}$, actions $a \in \mathcal{A}$, transitions $T \in \Delta(\mathcal{S})^{\mathcal{S} \times \mathcal{A}}$, rewards $R \in \mathbb{R}^{\mathcal{S} \times \mathcal{A}}$, and discount γ . Let $\pi \in \Delta(\mathcal{A})^{\mathcal{S}}$ denote a policy, with induced occupancy measure $\rho_\pi(s, a) \doteq \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t \mathbb{1}_{\{s_t=s, a_t=a\}}]$, where the expectation is understood to be taken over $a_t \sim \pi(\cdot|s_t)$ and $s_{t+1} \sim T(\cdot|s_t, a_t)$ from some initial distribution. We shall also write $\rho_\pi(s) \doteq \sum_a \rho_\pi(s, a)$ to indicate the state-only occupancy measure. In this paper, we operate in the most minimal setting where neither the environment dynamics nor the reward function is known. Classically, *imitation learning* [1–3] seeks an imitator policy π as follows:

$$\operatorname{argmin}_\pi \mathbb{E}_{s \sim \rho_\pi} \mathcal{L}(\pi_D(\cdot|s), \pi(\cdot|s)) \quad (1)$$

where $\mathcal{L}$ is some choice of loss. In practice, instead of π_D we are given access to a sampled dataset $\mathcal{D}$ of state-action pairs $s, a \sim \rho_D$. (While here we only assume access to *pairs*, some algorithms require *triples* that include next states). Behavioral cloning [4–6] is a well-known (but naive) approach that simply ignores the endogeneity of the rollout distribution, replacing ρ_π with ρ_D in the expectation. This reduces imitation learning to a supervised classification problem (popularly, with cross-entropy loss), though the potential disadvantage of disregarding the visitation distribution is well-studied [7–9].

Apprenticeship Learning To incorporate awareness of dynamics, a family of techniques (commonly referenced under the “apprenticeship learning” umbrella) have been developed, including classic inverse reinforcement learning algorithms and more recent methods in adversarial imitation learning. Note that the vast majority of these approaches are *online* in nature, though it is helpful for us to start with the same formalism. Consider the (maximum entropy) reinforcement learning setting, and let $R_t \doteq R(s_t, a_t)$ and $\mathcal{H}_t \doteq -\log \pi(\cdot|s_t)$. The (forward) primitive RL : $\mathbb{R}^{\mathcal{S} \times \mathcal{A}} \rightarrow \Delta(\mathcal{A})^{\mathcal{S}}$ is given by:

$$\text{RL}(R) \doteq \operatorname{argmax}_\pi \left(\mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t R_t] + H(\pi) \right) \quad (2)$$

where (as before) the expectation is understood to be taken with respect to π and the environment dynamics, and $H(\pi) \doteq \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t \mathcal{H}_t]$. A basic result [10, 11] is that the (soft) Bellman operator is contractive, so its fixed point (hence the optimal policy) is unique. Now, let $\psi : \mathbb{R}^{\mathcal{S} \times \mathcal{A}} \rightarrow \mathbb{R}$ denote a reward function regularizer. Then the (inverse) primitive IRL $_\psi : \Delta(\mathcal{A})^{\mathcal{S}} \rightarrow \mathcal{P}(\mathbb{R}^{\mathcal{S} \times \mathcal{A}})$ is given by:

$$\text{IRL}_\psi(\pi_D) \doteq \operatorname{argmin}_R \left(\psi(R) + \max_\pi \left(\mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t R_t] + H(\pi) \right) - \mathbb{E}_{\pi_D}[\sum_{t=0}^{\infty} \gamma^t R_t] \right) \quad (3)$$

Finally, let $\tilde{R} \in \text{IRL}_\psi(\pi_D)$ and $\pi = \text{RL}(\tilde{R})$, and denote by $\psi^* : \mathbb{R}^{\mathcal{S} \times \mathcal{A}} \rightarrow \mathbb{R}$ the Fenchel conjugate of regularizer ψ . A fundamental result [12] is that (ψ -regularized) apprenticeship learning can be taken as the composition of forward and inverse procedures, and obtains an imitator policy π such that the induced occupancy measure ρ_π is close to ρ_D as determined by the (convex) function ψ^* :

$$\text{RL} \circ \text{IRL}_\psi(\pi_D) = \operatorname{argmax}_\pi \left(-\psi^*(\rho_\pi - \rho_D) + H(\pi) \right) \quad (4)$$

Classically, IRL-based apprenticeship methods [13–21] simply execute RL repeatedly in an inner loop, with fixed regularizers ψ for tractability (such as indicators for linear and convex function classes). More recently, adversarial imitation learning techniques leverage Equation 4 (modulo $H(\pi)$, which is generally less important in practice), instantiating ψ^* with various ϕ -divergences [12, 22–27] and integral probability metrics [28, 29], thereby matching occupancy measures without unnecessary bias.

Strictly Batch Imitation Learning Unfortunately, advances in both IRL-based and adversarial IL have been developed with a very much *online* audience in mind: Precisely, their execution involves repeated on-policy rollouts, which requires access to an environment (for interaction), or at least knowledge of its dynamics (for simulation). Imitation learning in a completely *offline* setting provides neither. On the other hand, while behavioral cloning is “offline” to begin with, it is fundamentally limited by disregarding valuable (distributional) information in the demonstration data. Proposed

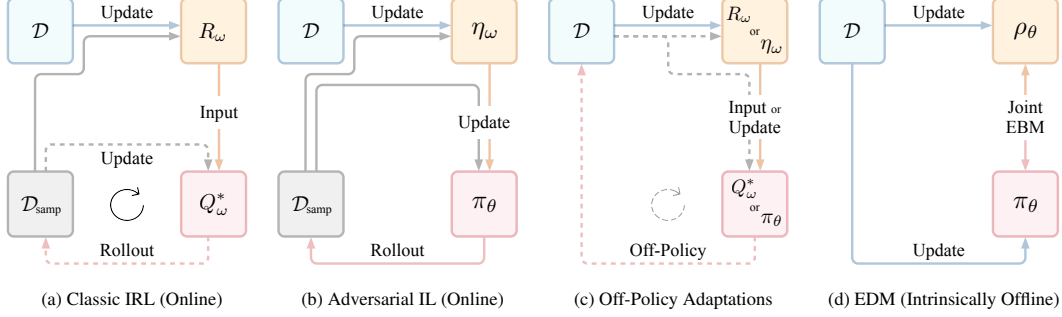


Figure 1: *From Online to Offline Learning.* (a) Classic IRL-based algorithms execute RL repeatedly in an inner loop, learning imitator policies indirectly via parameterizations ω of a reward function R_ω . (b) Adversarial IL methods seek a distribution-matching objective, alternately optimizing a policy π_θ parameterized by θ and a discriminator-like function η_ω (which in some cases can be taken as R or a value-function) parameterized by ω . (c) For strictly batch IL, one solution is simply to retrofit existing algorithms from (a) or (b) to work without any sampling actually taking place; this involves using off-policy evaluation as a workaround for these (intrinsically online) apprenticeship methods, which may introduce more variance than desired. (d) We propose a simpler but effective offline method by jointly learning a policy function with an energy-based model of the state distribution.

rectifications are infeasible, as they typically require querying the demonstrator, interacting with the environment, or knowledge of model dynamics or sparsity of rewards [30–33]. Now of course, an immediate question is whether existing apprenticeship methods can be more-or-less repurposed for batch learning (see Figure 1). The answer is certainly yes—but they might not be the most satisfying:

Adapting Classic IRL. Briefly, this would inherit the theoretical and computational disadvantages of classic IRL, plus additional difficulties from adapting to batch settings. First, IRL learns imitator policies slowly and indirectly via intermediate parameterizations of R , relying on repeated calls to a (possibly imperfect) inner RL procedure. Explicit constraints for tractability also mean that true rewards will likely be imperfectly captured without excessive feature engineering. Most importantly, batch IRL requires *off-policy* evaluation at every step—which is itself a nontrivial problem with imperfect solutions. For instance, for the max-margin, minimax, and max-likelihood approaches, adaptations for batch imitation [34–37] rely on least-squares TD and Q-learning, as well as depending on restrictions to linear rewards. Similarly, adaptations of policy-loss and Bayesian IRL in [34, 38] fall back on linear score-based classification and LSTD. Alternative workarounds involve estimating a model from demonstrations alone [39, 40]—feasible only for the smallest or discrete state spaces.

Adapting Adversarial IL. Analogously, the difficulty here is that the adversarial formulation requires expectations over trajectories sampled from imitator policy rollouts. Now, there has been recent work focusing on enabling *off-policy* learning through the use of off-policy actor-critic methods [41, 42]. However, this is accomplished by skewing the divergence minimization objective to minimize the distance between the distributions induced by the demonstrator and the replay buffer (instead of the imitator); they must still operate in an online fashion, and are not applicable in a strictly batch setting. More recently, a reformulation in [43] does away with a separate critic by learning the (log density ratio) “ Q -function” via the same objective used for distribution matching. While this theoretically enables fully offline learning, it inherits a similarly complex alternating max-min optimization procedure; moreover, the objective involves the logarithm of an expectation over an exponentiated difference in the Bellman operator—for which mini-batch approximations of gradients are biased.

Three Desiderata At risk of belaboring, the offline setting means that we already have *all* of the information we will ever get, right at the very start. Hanging on to the RL-centric structure of these intrinsically online apprenticeship methods relies entirely on off-policy techniques—which may introduce more variance than we can afford. In light of the preceding discussion, it is clear that a good solution to the strictly batch imitation learning (SBIL) problem should satisfy the following criteria:

1. **Policy:** First, it should directly learn a policy (capturing “stepwise” action conditionals) without relying on learning intermediate rewards, and without generic constraints biasing the solution.
2. **Occupancy:** But unlike the (purely discriminative) nature of behavioral cloning, it should (generatively) account for information from rollout distributions (capturing “global” state marginals).
3. **Intrinsically Batch:** Finally, it should work offline without known/learned models, and without resorting to off-policy evaluations done within inner loops/max-min optimizations (see Table 1).

3 Energy-based Distribution Matching

We begin by parameterizing with θ our policy π_θ , and occupancy measure ρ_θ . We are interested in (explicitly) learning a policy while (implicitly) minimizing a divergence between occupancy measures:

$$\operatorname{argmin}_\theta D_\phi(\rho_D \parallel \rho_\theta) \quad (5)$$

for some choice of generator ϕ . Note that, unlike in the case of online apprenticeship, our options are significantly constrained by the fact that rollouts of π_θ are not actually possible. In the sequel, we shall use $\phi(u) = u \log u$, which gives rise to the (forward) KL, so we write $\operatorname{argmin}_\theta D_{\text{KL}}(\rho_D \parallel \rho_\theta) = \operatorname{argmin}_\theta -\mathbb{E}_{s,a \sim \rho_D} \log \rho_\theta(s, a)$. Now, consider the general class of stationary policies of the form:

$$\pi_\theta(a|s) = \frac{e^{f_\theta(s)[a]}}{\sum_a e^{f_\theta(s)[a]}} \quad (6)$$

where $f_\theta : \mathcal{S} \rightarrow \mathbb{R}^A$ indicates the logits for action conditionals. An elementary result [44, 45] shows a bijective mapping between the space of policies and occupancy measures satisfying the Bellman flow constraints, and $\pi(a|s) = \rho_\pi(s, a) / \rho_\pi(s)$; this allows decomposing the log term in the divergence as:

$$\log \rho_\theta(s, a) = \log \rho_\theta(s) + \log \pi_\theta(a|s) \quad (7)$$

Objective Ideally, our desired loss is therefore:

$$\mathcal{L}(\theta) = -\mathbb{E}_{s \sim \rho_D} \log \rho_\theta(s) - \mathbb{E}_{s,a \sim \rho_D} \log \pi_\theta(a|s) \quad (8)$$

with the corresponding gradient given by:

$$\nabla_\theta \mathcal{L}(\theta) = -\mathbb{E}_{s \sim \rho_D} \nabla_\theta \log \rho_\theta(s) - \mathbb{E}_{s,a \sim \rho_D} \nabla_\theta \log \pi_\theta(a|s) \quad (9)$$

Now, there is an obvious problem. Backpropagating through the first term is impossible as we cannot compute $\rho_\theta(s)$ —nor do we have access to online rollouts of π_θ to explicitly estimate it. In this offline imitation setting, our goal is to answer the question: Is there any benefit in *learning* an approximate model in its place instead? Here we consider energy-based modeling [46], which associates scalar measures of compatibility (i.e. energies) with configurations of variables (i.e. states). Specifically, we take advantage of the *joint energy-based modeling* approach [47–49]—in particular the proposal for a classifier to be simultaneously learned with a density model defined implicitly by the logits of the classifier (which—as they observe—yields improvements such as in calibration and robustness):

Joint Energy-based Modeling Consider first the general class of energy-based models (EBMs) for state occupancy measures $\rho_\theta(s) \propto e^{-E(s)}$. Now, mirroring the exposition in [47], note that a model of the state-action occupancy measure $\rho_\theta(s, a) = e^{f_\theta(s)[a]} / Z_\theta$ can be defined via the parameterization for π_θ , where Z_θ is the partition function. The state-only model for $\rho_\theta(s) = \sum_a e^{f_\theta(s)[a]} / Z_\theta$ is then obtained by marginalizing out a . In other words, the parameterization of π_θ already implicitly defines an EBM of state visitation distributions with the energy function $E_\theta : \mathbb{R}^{|\mathcal{S}|} \rightarrow \mathbb{R}^{|\mathcal{A}|}$ given as follows:

$$E_\theta(s) \doteq -\log \sum_a e^{f_\theta(s)[a]} \quad (10)$$

The chief difference from [47], of course, is that here the true probabilities in question are not static class conditionals/marginals: The *actual* occupancy measure corresponds to rolling out π_θ , and if we could do that, we would naturally recover an approach not unlike the variety of distribution-aware algorithms in the literature; see e.g. [50]. In the strictly batch setting, we clearly cannot sample directly from this (online) distribution. However, as a matter of multitask learning, we still hope to gain from jointly learning an (offline) *model* of the state distribution—which we can then freely sample from:

Proposition 1 (Surrogate Objective) Define the “occupancy” loss $\mathcal{L}_\rho$ as the difference in energy:

$$\mathcal{L}_\rho(\theta) \doteq \mathbb{E}_{s \sim \rho_D} E_\theta(s) - \mathbb{E}_{s \sim \rho_\theta} E_\theta(s) \quad (11)$$

Then $\nabla_\theta \mathcal{L}_\rho(\theta) = -\mathbb{E}_{s \sim \rho_D} \nabla_\theta \log \rho_\theta(s)$. In other words, differentiating this recovers the first term in Equation 9. Therefore if we define a standard “policy” loss $\mathcal{L}_\pi(\theta) \doteq -\mathbb{E}_{s,a \sim \rho_D} \log \pi_\theta(a|s)$, then:

$$\mathcal{L}_{\text{sur}}(\theta) \doteq \mathcal{L}_\rho(\theta) + \mathcal{L}_\pi(\theta) \quad (12)$$

yields a surrogate objective that can be optimized, instead of the original $\mathcal{L}$. Note that by relying on the offline energy-based model, we now have access to the gradients of the terms in the expectations.

Algorithm 1 Energy-based Distribution Matching ▷ for Strictly Batch Imitation Learning

```

1: Input: SGLD hyperparameters  $\alpha, \sigma$ , PCD hyperparameters  $\kappa, \iota, \delta$ , and mini-batch size  $N$ 
2: Initialize: Policy network parameters  $\theta$ , and PCD buffer  $B_\kappa$ 
3: while not converged do
4:   Sample  $(s_1, a_1), \dots, (s_N, a_N) \sim \mathcal{D}$  from demonstrations dataset
5:   Sample  $(\tilde{s}_{1,0}, \dots, \tilde{s}_{N,0})$  as  $\tilde{s}_{n,0} \sim B_\kappa$  w.p.  $1 - \delta$  o.w.  $\tilde{s}_{n,0} \sim \mathcal{U}(\mathcal{S})$ 
6:   for  $i = 1, \dots, \iota$  do
7:      $\tilde{s}_{n,i} = \tilde{s}_{n,i-1} - \alpha \cdot \partial E_\theta(\tilde{s}_{n,i-1}) / \partial \tilde{s}_{n,i-1} + \sigma \cdot \mathcal{N}(0, I), \forall n \in \{1, \dots, N\}$ 
8:      $\hat{\mathcal{L}}_\pi \leftarrow \frac{1}{N} \sum_{n=1}^N \text{CrossEntropy}(\pi_\theta(\cdot | s_n), a_n)$  ▷  $\mathcal{L}_\pi = -\mathbb{E}_{s, a \sim \rho_D} \log \pi_\theta(a | s)$ 
9:      $\hat{\mathcal{L}}_\rho \leftarrow \frac{1}{N} \sum_{n=1}^N E_\theta(s_n) - \frac{1}{N} \sum_{n=1}^N E_\theta(\tilde{s}_{n,\iota})$  ▷  $\mathcal{L}_\rho = \mathbb{E}_{s \sim \rho_D} E_\theta(s) - \mathbb{E}_{s \sim \rho_\theta} E_\theta(s)$ 
10:    Add  $\tilde{s}_{n,\iota}$  to  $B_\kappa, \forall n \in \{1, \dots, N\}$ 
11:    Backpropagate  $\nabla_\theta \hat{\mathcal{L}}_\rho + \nabla_\theta \hat{\mathcal{L}}_\pi$ 
12: Output: Learned policy parameters  $\theta$ 

```

Proof. Appendix A. Sketch: For any s , write $\rho_\theta(s) = e^{-E_\theta(s)} / \int_{\mathcal{S}} e^{-E_\theta(s)} ds$, for which the gradient of the logarithm is given by $-\nabla_\theta \log \rho_\theta(s) = \nabla_\theta E_\theta(s) - \mathbb{E}_{s \sim \rho_\theta} \nabla_\theta E_\theta(s)$. Then, taking expectations over ρ_D and substituting in the energy term as given by Equation 10, straightforward manipulation shows $-\nabla_\theta \mathbb{E}_{s \sim \rho_D} \log \rho_\theta(s) = \nabla_\theta \mathcal{L}_\rho(\theta)$. The second part follows immediately from Equation 8. $\square$

Why is this better than before? Because using the original objective $\mathcal{L}$ required us to know $\rho_\theta(s)$, which—even modeled separately as an EBM—we do not (since we cannot compute the normalizing constant). On the other hand, using the surrogate objective $\mathcal{L}_{\text{sur}}$ only requires being able to sample from the EBM, which is easier. Note that jointly learning the EBM does not constrain/bias the policy, as this simply reuses the policy parameters along with the extra degree of freedom in the logits $f_\theta(s)[\cdot]$.

Optimization The EDM surrogate objective entails minimal addition to the standard behavioral cloning loss. Accordingly, it is perfectly amenable to mini-batch gradient approximations—unlike for instance [43], for which mini-batch gradients are biased in general. We approximate the expectation over ρ_θ in Equation 11 via stochastic gradient Langevin dynamics (SGLD) [51], which follows recent successes in training EBMs parameterized by deep networks [47, 48, 52], and use persistent contrastive divergence (PCD) [53] for computational savings. Specifically, each sample is drawn as:

$$\tilde{s}_i = \tilde{s}_{i-1} - \alpha \cdot \frac{\partial E_\theta(\tilde{s}_{i-1})}{\partial \tilde{s}_{i-1}} + \sigma \cdot \mathcal{N}(0, I) \quad (13)$$

where α denotes the SGLD learning rate, and σ the noise coefficient. Algorithm 1 details the EDM optimization procedure, with a buffer B_κ of size κ , reinitialization frequency δ , and number of iterations ι , where $\tilde{s}_0 \sim \rho_0(s)$ is sampled uniformly. Note that the buffer here should not be confused with the “replay buffer” within (online) imitation learning algorithms, to which it bears no relationship whatsoever. In practice, we find that the configuration given in [47] works effectively with only small modifications. We refer to [46–49, 51, 53] for discussion of general considerations for EBM optimization.

4 Analysis and Interpretation

Our development in Section 3 proceeded in three steps. First, we set out with a divergence minimization objective in mind (Equation 5). With the aid of the decomposition in Equation 7, we obtained the original (online) maximum-likelihood objective function (Equation 8). Finally, using Proposition 1, we instead optimize an (offline) joint energy-based model by scaling the gradient of a surrogate objective (Equation 12). Now, the mechanics of the optimization are straightforward, but what is the underlying motivation for doing so? In particular, how does the EDM objective relate to existing notions of (1) divergence minimization, (2) joint learning, as well as (3) imitation learning in general?

Divergence Minimization With the seminal observation by [12] of the equivalence in Equation 4, the IL arena was quickly populated with a lineup of adversarial algorithms minimizing a variety of distances [25–29], and the forward KL in this framework was first investigated in [27]. However in the strictly batch setting, we have no ability to compute (or even sample from) the actual rollout distribution for π_θ , so we instead choose to learn an EBM in its place. To be clear, we are now doing something quite different than [25–29]: In minimizing the divergence (Equation 5) by simultaneously learning an (offline) model instead of sampling from (online) rollouts, π_θ and ρ_θ are no longer coupled in terms of rollout *dynamics*, and the coupling that remains is in terms of the underlying *parameterization* θ . That is the price we pay. At the same time, hanging on to the adversarial setup in the batch setting requires

Table 1: *From Online to Offline Imitation*. Recall the three desiderata from Section 2, where we seek an SBIL solution that: **(1)** learns a *directly parameterized* policy, without restrictive constraints biasing the solution—e.g. restrictions to linear/convex function classes for intermediate rewards, or generic norm-based penalties on reward sparsity; **(2)** is *dynamics-aware* by accounting for distributional information—either through temporal or parameter consistency; and **(3)** is *intrinsically batch*, in the sense of being operable strictly offline, and directly optimizable—i.e. without recourse to off-policy evaluations in costly inner loops or alternating max-min optimizations.

	Formulation	Example	Parameterized Policy ⁽¹⁾	Non-Restrictive Regularizer ⁽¹⁾	Dynamics Awareness ⁽²⁾	Operable Strictly Batch ⁽³⁾	Directly Optimized ⁽³⁾
Online (Original)	Max Margin	[13, 14]	×	×	✓	×	×
	Minimax Game	[17]	×	×	✓	×	×
	Min Policy Loss	[15]	×	×	✓	×	×
	Max Likelihood	[19]	×	×	✓	×	×
	Max Entropy	[10, 18]	×	×	✓	×	×
	Max A Posteriori	[16, 20]	×	×	✓	×	×
	Adversarial Imitation	[12, 22–27]	✓	✓	✓	×	×
Off. (Adaptation)	Max Margin	[34, 37]	×	×	✓	✓	×
	Minimax Game	[35]	×	×	✓	✓	×
	Min Policy Loss	[54]	×	×	✓	✓	✓
	Max Likelihood	[36]	×	×	✓	✓	×
	Max Entropy	[39]	×	×	✓	✓	×
	Max A Posteriori	[38]	×	×	✓	✓	✓
	Adversarial Imitation	[43]	✓	✓	✓	✓	×
Behavioral Cloning			✓	×	×	✓	✓
Reward-Regularized BC			✓	×	✓	✓	✓
EDM			✓	✓	✓	✓	✓
(Ours)			✓	✓	✓	✓	✓

estimating intrinsically on-policy terms via off-policy methods, which are prone to suffer from high variance. Moreover, the divergence minimization interpretations of adversarial IL hinge crucially on the assumption that the discriminator-like function is perfectly optimized [12, 25, 27, 43]—which may not be realized in practice offline. The EDM objective aims to sidestep both of these difficulties.

Joint Learning In the online setting, minimizing Equation 8 is equivalent to injecting temporal consistency into behavioral cloning: While the $\mathbb{E}_{s,a \sim \rho_D} \log \pi_\theta(a|s)$ term is purely a discriminative objective, the $\mathbb{E}_{s \sim \rho_D} \log \rho_\theta(s)$ term additionally constrains $\pi_\theta(\cdot|s)$ to the space of policies for which the induced state distribution matches the data. In the offline setting, instead of this *temporal* relationship we are now leveraging the *parameter* relationship between π_θ and ρ_θ —that is, from the joint EBM. In effect, this accomplishes an objective similar to multitask learning, where representations of both discriminative (policy) and generative (visitation) distributions are learned by sharing the same underlying function approximator. As such, (details of sampling techniques aside) this additional mandate does *not* add any bias. This is in contrast to generic approaches to regularization in IL, such as the norm-based penalties on the sparsity of implied rewards [9, 32, 55]—which adds bias. The state-occupancy constraint in EDM simply harnesses the extra degree of freedom hidden in the logits $f_\theta(s)$ —which are normally allowed to shift by an arbitrary scalar—to define the density over states.

Imitation Learning Finally, recall the classical notion of *imitation learning* that we started with (Equation 1). As noted earlier, naive application by behavioral cloning simply ignores the endogeneity of the rollout distribution. How does our final surrogate objective (Equation 12) relate to this? First, we place Equation 1 in the maximum entropy RL framework in order to speak in a unified language:

Proposition 2 (Classical Objective) Consider the classical IL objective in Equation 1, with policies parameterized as Equation 6. Choosing $\mathcal{L}$ to be the (forward) KL divergence yields the following:

$$\operatorname{argmax}_R (\mathbb{E}_{s \sim \rho_R^*} \mathbb{E}_{a \sim \pi_D(\cdot|s)} Q_R^*(s, a) - \mathbb{E}_{s \sim \rho_R^*} V_R^*(s)) \quad (14)$$

where $Q_R^* : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the (soft) Q -function given by $Q_R^*(s, a) = R(s, a) + \gamma \mathbb{E}_T[V_R^*(s')|s, a]$, $V^*(s) \in \mathbb{R}^{\mathcal{S}}$ is the (soft) value function $V_R^*(s) = \log \sum_a e^{Q_R^*(s, a)}$, and ρ_R^* is the occupancy for π_R^* .

Proof. Appendix A. This relies on the fact that we are free to identify the logits f_θ of our policy with a (soft) Q -function. Specifically, this requires the additional fact that the mapping between Q -functions and reward functions is bijective, which we also state (and prove) as Lemma 5 in Appendix A. $\square$

This is intuitive: It states that classical imitation learning with $\mathcal{L} = D_{\text{KL}}$ is equivalent to searching for a reward function R . In particular, we are looking for an R for which—in expectation over rollouts of policy π_R^* —the advantage function $Q_R^*(s, a) - V_R^*(s)$ for taking actions $a \sim \pi_D(\cdot|s)$ is

maximal. Now, the following distinction is key: While Equation 14 is perfectly valid as a choice of objective, it is a certain (naive) substitution in the offline setting that is undesirable. Specifically, Equation 14 is precisely what behavioral cloning attempts to do, but—without the ability to perform π_R^* rollouts—it simply replaces ρ_R^* with ρ_D . This is *not* an (unbiased) “approximation” in, say, the sense that $\hat{\mathcal{L}}_\rho$ empirically approximates $\mathcal{L}_\rho$, and is especially inappropriate when ρ_D contains very few demonstrations to begin with. While EDM cannot fully “undo” the damage (nothing can do that in the strictly batch setting), it uses a “smoothed” EBM in place of ρ_D , which—as we shall see empirically—leads to largest improvements precisely when the number of demonstrations are few.

Proposition 3 (From BC to EDM) The behavioral cloning objective is equivalently the following, where—compared to Equation 14—expectations over states are now taken w.r.t. ρ_D instead of ρ_R^* :

$$\operatorname{argmax}_R \left(\mathbb{E}_{s \sim \rho_D} \mathbb{E}_{a \sim \pi_D(\cdot|s)} Q_R^*(s, a) - \mathbb{E}_{s \sim \rho_D} V_R^*(s) \right) \quad (15)$$

In contrast, by augmenting the (behavioral cloning) “policy” loss $\mathcal{L}_\pi$ with the “occupancy” loss $\mathcal{L}_\rho$, what the EDM surrogate objective achieves is to replace one of the expectations with the learned ρ_θ :

$$\operatorname{argmax}_R \left(\mathbb{E}_{s \sim \rho_D} \mathbb{E}_{a \sim \pi_D(\cdot|s)} Q_R^*(s, a) - \mathbb{E}_{s \sim \rho_\theta} V_R^*(s) \right) \quad (16)$$

Proof. Appendix A. The reasoning for both statements follows a similar form as for Proposition 2. $\square$

Note that by swapping out ρ_R^* for ρ_D in behavioral cloning, the (dynamics) relationship between π_R^* and its induced occupancy measure is (completely) broken, and the optimization in Equation 15 is equivalent to performing a sort of inverse reinforcement learning with no constraints whatsoever on R . What the EDM surrogate objective does is to “repair” one of the expectations to allow sampling from a smoother model distribution ρ_θ than the (possibly very sparse) data distribution ρ_D . (Can we also “repair” the other term? But this is now asking to somehow warp $\mathbb{E}_{s \sim \rho_D} \mathbb{E}_{a \sim \pi_D(\cdot|s)} Q_R^*(s, a)$ into $\mathbb{E}_{s \sim \rho_\theta} \mathbb{E}_{a \sim \pi_D(\cdot|s)} Q_R^*(s, a)$. All else equal, this is certainly impossible without querying the expert.)

5 Experiments

Benchmarks We test Algorithm 1 (**EDM**) against the following benchmarks, varying the amount of demonstration data $\mathcal{D}$ (from a single trajectory to 15) to illustrate sample complexity: The intrinsically offline behavioral cloning (**BC**), and reward-regularized classification (**RCAL**) [32]—which proposes to leverage dynamics information through a sparsity-based penalty on the implied rewards; the deep successor feature network (**DSFN**) algorithm of [37]—which is an off-policy adaptation of the max-margin IRL algorithm and a (deep) generalization of earlier (linear) approaches by LSTD [34, 38]; and the state-of-the-art in sample-efficient adversarial imitation learning (**VDICE**) in [43], which—while designed with an online audience in mind—can theoretically operate in a completely offline manner. (Remaining candidates in Table 1 are inapplicable, since they either only operate in discrete states [36, 39], or only output a reward [54], which—in the strictly batch setting—does not yield a policy.

Demonstrations We conduct experiments on control tasks and a real-world healthcare dataset. For the former, we use OpenAI gym environments [56] of varying complexity from standard RL literature: CartPole, which balances a pendulum on a frictionless track [57], Acrobot, which swings a system of joints up to a given height [58], BeamRider, which controls an Atari 2600 arcade space shooter [59], as well as LunarLander, which optimizes a rocket trajectory for successful landing [60]. Demonstration datasets $\mathcal{D}$ are generated using pre-trained and hyperparameter-optimized agents from the RL Baselines Zoo [61] in Stable OpenAI Baselines [62]. For the healthcare application, we use MIMIC-III, a real-world medical dataset consisting of patients treated in intensive care units from the Medical Information Mart for Intensive Care [63], which records trajectories of physiological states and treatment actions (e.g. antibiotics and ventilator support) for patients at one-day intervals.

Implementation The experiment is arranged as follows: Demonstrations $\mathcal{D}$ are sampled for use as input to train all algorithms, which are then evaluated using 300 live episodes (for OpenAI gym environments) or using a held-out test set (for MIMIC-III). This process is then repeated for a total 50 times (using different $\mathcal{D}$ and randomly initialized networks), from which we compile the means of the performances (and their standard errors) for each algorithm. Policies trained by all algorithms share the same network architecture: two hidden layers of 64 units each with ELU activation (or—for Atari—three convolutional layers with ReLU activation). For DSFN, we use the publicly available source code at [64], and likewise for VDICE, which is available at [65]. Note that VDICE is originally designed for Gaussian actions, so we replace the output layer of the actor with a Gumbel-softmax parameterization; offline learning is enabled by setting the “replay regularization” coefficient to zero.

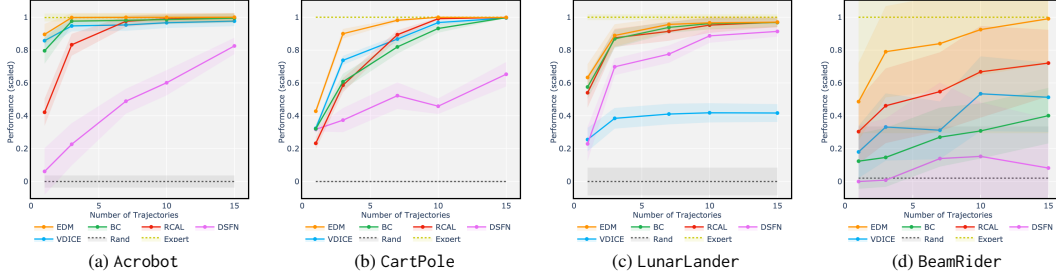


Figure 2: *Performance Comparison for Gym Environments*. The x -axis indicates the amount of demonstration data provided (i.e. number of trajectories, in $\{1, 3, 7, 10, 15\}$), and the y -axis shows the average returns of each imitation algorithm (scaled so that the demonstrator attains a return of 1 and a random policy network attains 0).

	2-Action Setting (Ventilator Only)			4-Action Setting (Antibiotics + Vent.)		
Metrics	ACC	AUC	APR	ACC	AUC	APR
BC	0.861 ± 0.013	0.914 ± 0.003	0.902 ± 0.005	0.696 ± 0.006	0.859 ± 0.003	0.659 ± 0.007
RCAL	0.872 ± 0.007	0.911 ± 0.007	0.898 ± 0.006	0.701 ± 0.007	0.864 ± 0.003	0.667 ± 0.006
DSFN	0.865 ± 0.007	0.906 ± 0.003	0.885 ± 0.001	0.682 ± 0.005	0.857 ± 0.002	0.665 ± 0.003
VDICE	0.875 ± 0.004	0.915 ± 0.001	0.904 ± 0.002	0.707 ± 0.005	0.864 ± 0.002	0.673 ± 0.003
Rand	0.498 ± 0.007	0.500 ± 0.000	0.500 ± 0.000	0.251 ± 0.005	0.500 ± 0.000	0.250 ± 0.000
EDM	0.891 ± 0.004	0.922 ± 0.004	0.912 ± 0.005	0.720 ± 0.007	0.873 ± 0.002	0.681 ± 0.008

Table 2: *Performance Comparison for MIMIC-III*. Action-matching is used to assess the quality of clinical policies learned in both the 2-action and 4-action settings. We report the accuracy of action selection (ACC), the area under the receiving operator characteristic curve (AUC), and the area under the precision-recall curve (APR).

Algorithm 1 is implemented using the source code for joint EBMs [47] publicly available at [66], which already contains an implementation of SGLD. Note that the only difference between BC and EDM is the addition of $\mathcal{L}_\rho$, and the RCAL loss is straightforwardly obtained by inverting the Bellman equation. See Appendix B for additional detail on experiment setup, benchmarks, and environments.

Evaluation and Results For gym environments, the performance of trained imitator policies (learned offline) is evaluated with respect to (true) *average returns* generated by deploying them live. Figure 2 shows the results for policies given different numbers of trajectories as input to training, and Appendix B provides exact numbers. For the MIMIC-III dataset, policies are trained and tested on demonstrations by way of cross-validation; since we have no access to ground-truth rewards, we assess performance according to *action-matching* on held-out test trajectories, per standard [64]; Table 2 shows the results. With respect to either metric, we find that EDM consistently produces policies that perform similarly or better than benchmark algorithms in all environments, especially in low-data regimes. Also notable is that in this strictly batch setting (i.e. where no online sampling whatsoever is permitted), the off-policy adaptations of online algorithms (i.e. DSFN, VDICE) do not perform as consistently as the intrinsically offline ones—especially DSFN, which involves predicting entire next states (off-policy) for estimating feature maps; this validates some of our original motivations. Finally, note that—via the joint EBM—the EDM algorithm readily accommodates (semi-supervised) learning from additional state-only data (with unobserved actions); additional result in Appendix B.

6 Discussion

In this work, we motivated and presented EDM for strictly batch imitation, which retains the simplicity of direct policy learning while accounting for information in visitation distributions. However, we are sampling from an offline model (leveraging multitask learning) of state visitations, not from actual online rollouts (leveraging temporal consistency), so they can only be so useful. The objective also relies on the assumption that samples in $\mathcal{D}$ are sufficiently representative of ρ_D ; while this is standard in literature [42], it nonetheless bears reiteration. Our method is agnostic as to discrete/continuous state spaces, but the use of joint EBMs means we only consider categorical actions in this work. That said, the application of EBMs to regression is increasingly of focus [67], and future work may investigate the possibility of extending EDM to continuous actions. Overall, our work is enabled by recent advances in joint EBMs, and similarly use contrastive divergence to approximate the KL gradient. Note that EBMs in general may not be the easiest to train, or to gauge learning progress for [47]. However, for the types of environments we consider, we did not find stability-related issues to be nearly as noticeable as typical of the higher-dimensional imaging tasks EBMs are commonly used for.

Broader Impact

In general, any method for imitation learning has the potential to mitigate problems pertaining to scarcity of expert knowledge and computational resources. For instance, consider a healthcare institution strapped for time and personnel attention—such as one under the strain of an influx of ICU patients. If implemented as a system for clinical decision support and early warnings, even the most bare-bones policy trained on optimal treatment/monitoring actions has huge potential for streamlining medical decisions, and for allocating attention where real-time clinical judgment is most required.

By focusing our work on the strictly batch setting for learning, we specifically accommodate situations that disallow directly experimenting on the environment during the learning process. This consideration is critical in many conceivable applications: In practice, humans are often on the receiving end of actions and policies, and an imitator policy that must learn by interactive experimentation would be severely hampered due to considerations of cost, danger, or moral hazard. While—in line with literature—we illustrate the technical merits of our proposed method with respect to standard control environments, we do take care to highlight the broader applicability of our approach to healthcare settings, as it likewise applies—without saying—to education, insurance, or even law enforcement.

Of course, an important caveat is that any method for imitation learning naturally runs the risk of internalizing any existing human biases that may be implicit in the demonstrations collected as training input. That said, a growing field in reinforcement learning is dedicated to maximizing interpretability in learned policies, and—in the interest of accountability and transparency—striking an appropriate balance with performance concerns will be an interesting direction of future research.

Acknowledgments

We would like to thank the reviewers for their generous and invaluable comments and suggestions. This work was supported by Alzheimer’s Research UK (ARUK), The Alan Turing Institute (ATI) under the EPSRC grant EP/N510129/1, The US Office of Naval Research (ONR), and the National Science Foundation (NSF) under grant numbers 1407712, 1462245, 1524417, 1533983, and 1722516.

References

- [1] Hoang M Le, Andrew Kang, Yisong Yue, and Peter Carr. Smooth imitation learning for online sequence prediction. *International Conference on Machine Learning (ICML)*, 2016.
- [2] Ahmed Hussein, Mohamed Medhat Gaber, Eyad Elyan, and Chrisina Jayne. Imitation learning: A survey of learning methods. *ACM Computing Surveys (CSUR)*, 2017.
- [3] Yisong Yue and Hoang M Le. Imitation learning (presentation). *International Conference on Machine Learning (ICML)*, 2018.
- [4] Dean A Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural computation (NC)*, 1991.
- [5] Michael Bain and Claude Sammut. A framework for behavioural cloning. *Machine Intelligence (MI)*, 1999.
- [6] Umar Syed and Robert E Schapire. A reduction from apprenticeship learning to classification. *Advances in neural information processing systems (NeurIPS)*, 2010.
- [7] Stéphane Ross and Drew Bagnell. Efficient reductions for imitation learning. *International conference on artificial intelligence and statistics (AISTATS)*, 2010.
- [8] Francisco S Melo and Manuel Lopes. Learning from demonstration using mdp induced metrics. *Joint European conference on machine learning and knowledge discovery in databases (ECML)*, 2010.
- [9] Bilal Piot, Matthieu Geist, and Olivier Pietquin. Boosted and reward-regularized classification for apprenticeship learning. *International conference on Autonomous agents and multi-agent systems (AAMAS)*, 2014.
- [10] Brian D Ziebart. Modeling purposeful adaptive behavior with the principle of maximum causal entropy. *Phd Dissertation, Carnegie Mellon University*, 2010.

- [11] Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. *International Conference on Machine Learning (ICML)*, 2017.
- [12] Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. *Advances in neural information processing systems (NeurIPS)*, 2016.
- [13] Andrew Y Ng, Stuart J Russell, et al. Algorithms for inverse reinforcement learning. *International conference on Machine learning (ICML)*, 2000.
- [14] Pieter Abbeel and Andrew Y Ng. Apprenticeship learning via inverse reinforcement learning. *International conference on Machine learning (ICML)*, 2004.
- [15] Gergely Neu and Csaba Szepesvári. Apprenticeship learning using irl and gradient methods. *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2007.
- [16] Deepak Ramachandran and Eyal Amir. Bayesian inverse reinforcement learning. *International Joint Conference on Artificial Intelligence (IJCAI)*, 2007.
- [17] Umar Syed and Robert E Schapire. A game-theoretic approach to apprenticeship learning. *Advances in neural information processing systems (NeurIPS)*, 2008.
- [18] Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, and Anind K Dey. Maximum entropy inverse reinforcement learning. *AAAI Conference on Artificial Intelligence (AAAI)*, 2008.
- [19] Monica Babes, Vukosi Marivate, and Michael L Littman. Apprenticeship learning about multiple intentions. *International conference on Machine learning (ICML)*, 2011.
- [20] Jaedeug Choi and Kee-Eung Kim. Map inference for bayesian inverse reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2011.
- [21] Daniel Jarrett and Mihaela van der Schaar. Inverse active sensing: Modeling and understanding timely decision-making. *International Conference on Machine Learning*, 2020.
- [22] Nir Baram, Oron Anschel, and Shie Mannor. Model-based adversarial imitation learning. *International Conference on Machine Learning (ICML)*, 2017.
- [23] Wonseok Jeon, Seokin Seo, and Kee-Eung Kim. A bayesian approach to generative adversarial imitation learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [24] Chelsea Finn, Paul Christiano, Pieter Abbeel, and Sergey Levine. A connection between generative adversarial networks, inverse reinforcement learning, and energy-based models. *NeurIPS 2016 Workshop on Adversarial Training*, 2016.
- [25] Justin Fu, Katie Luo, and Sergey Levine. Learning robust rewards with adversarial inverse reinforcement learning. *International Conference on Learning Representations (ICLR)*, 2018.
- [26] Ahmed H Qureshi, Byron Boots, and Michael C Yip. Adversarial imitation via variational inverse reinforcement learning. *International Conference on Learning Representations (ICLR)*, 2019.
- [27] Seyed Kamyar Seyed Ghasemipour, Richard Zemel, and Shixiang Gu. A divergence minimization perspective on imitation learning methods. *Conference on Robot Learning (CoRL)*, 2019.
- [28] Kee-Eung Kim and Hyun Soo Park. Imitation learning via kernel mean embedding. *AAAI Conference on Artificial Intelligence (AAAI)*, 2018.
- [29] Huang Xiao, Michael Herman, Joerg Wagner, Sebastian Ziesche, Jalal Etesami, and Thai Hong Linh. Wasserstein adversarial imitation learning. *arXiv preprint*, 2019.
- [30] Umar Syed and Robert E Schapire. Imitation learning with a value-based prior. *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2007.
- [31] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *International conference on artificial intelligence and statistics (AISTATS)*, 2011.
- [32] Bilal Piot, Matthieu Geist, and Olivier Pietquin. Bridging the gap between imitation learning and irl. *IEEE transactions on neural networks and learning systems*, 2017.
- [33] Alexandre Attia and Sharone Dayan. Global overview of imitation learning. *arXiv preprint*, 2018.

- [34] Edouard Klein, Matthieu Geist, and Olivier Pietquin. Batch, off-policy and model-free apprenticeship learning. *European Workshop on Reinforcement Learning (EWRL)*, 2011.
- [35] Takeshi Mori, Matthew Howard, and Sethu Vijayakumar. Model-free apprenticeship learning for transfer of human impedance behaviour. *IEEE-RAS International Conference on Humanoid Robots*, 2011.
- [36] Vinamra Jain, Prashant Doshi, and Bikramjit Banerjee. Model-free irl using maximum likelihood estimation. *AAAI Conference on Artificial Intelligence (AAAI)*, 2019.
- [37] Donghun Lee, Srivatsan Srinivasan, and Finale Doshi-Velez. Truly batch apprenticeship learning with deep successor features. *International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.
- [38] Aristide CY Tossou and Christos Dimitrakakis. Probabilistic inverse reinforcement learning in unknown environments. *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2013.
- [39] Michael Herman, Tobias Gindele, Jörg Wagner, Felix Schmitt, and Wolfram Burgard. Inverse reinforcement learning with simultaneous estimation of rewards and dynamics. *International conference on artificial intelligence and statistics (AISTATS)*, 2016.
- [40] Ajay Kumar Tanwani and Aude Billard. Inverse reinforcement learning for compliant manipulation in letter handwriting. *National Center of Competence in Robotics (NCCR)*, 2013.
- [41] Lionel Blondé and Alexandros Kalousis. Sample-efficient imitation learning via gans. *International conference on artificial intelligence and statistics (AISTATS)*, 2019.
- [42] Ilya Kostrikov, Kumar Krishna Agrawal, Debidatta Dwibedi, Sergey Levine, and Jonathan Tompson. Discriminator-actor-critic: Addressing sample inefficiency and reward bias in adversarial imitation. *International Conference on Learning Representations (ICLR)*, 2019.
- [43] Ilya Kostrikov, Ofir Nachum, and Jonathan Tompson. Imitation learning via off-policy distribution matching. *International Conference on Learning Representations (ICLR)*, 2020.
- [44] Eugene A Feinberg and Adam Shwartz. *Markov decision processes: methods and applications*. Springer Science & Business Media, 2012.
- [45] Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.
- [46] Yann LeCun, Sumit Chopra, Raia Hadsell, M Ranzato, and F Huang. A tutorial on energy-based learning. *Predicting structured data*, 2006.
- [47] Will Grathwohl, Kuan-Chieh Wang, Jörn-Henrik Jacobsen, David Duvenaud, Mohammad Norouzi, and Kevin Swersky. Your classifier is secretly an energy based model and you should treat it like one. *International Conference on Learning Representations (ICLR)*, 2020.
- [48] Yilun Du and Igor Mordatch. Implicit generation and generalization in energy-based models. *Advances in neural information processing systems (NeurIPS)*, 2019.
- [49] Jianwen Xie, Yang Lu, Song-Chun Zhu, and Yingnian Wu. A theory of generative convnet. *International Conference on Machine Learning (ICML)*, 2016.
- [50] Yannick Schroecker and Charles L Isbell. State aware imitation learning. *Advances in neural information processing systems (NeurIPS)*, 2017.
- [51] Max Welling and Yee W Teh. Bayesian learning via stochastic gradient langevin dynamics. *International Conference on Machine Learning (ICML)*, 2011.
- [52] Erik Nijkamp, Mitch Hill, Tian Han, Song-Chun Zhu, and Ying Nian Wu. On the anatomy of mcmc-based maximum likelihood learning of energy-based models. *AAAI Conference on Artificial Intelligence (AAAI)*, 2020.
- [53] Tijmen Tieleman. Training restricted boltzmann machines using approximations to the likelihood gradient. *International Conference on Machine Learning (ICML)*, 2008.
- [54] Edouard Klein, Matthieu Geist, Bilal Piot, and Olivier Pietquin. Irl through structured classification. *Advances in neural information processing systems (NeurIPS)*, 2012.
- [55] Siddharth Reddy, Anca D Dragan, and Sergey Levine. Sqil: Imitation learning via regularized behavioral cloning. *International Conference on Learning Representations (ICLR)*, 2020.
- [56] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *OpenAI*, 2016.

- [57] Andrew G Barto, Richard S Sutton, and Charles W Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE transactions on systems, man, and cybernetics*, 1983.
- [58] Alborz Geramifard, Christoph Dann, Robert H Klein, William Dabney, and Jonathan P How. Rlpy: a value-function-based reinforcement learning framework for education and research. *Journal of Machine Learning Research (JMLR)*, 2015.
- [59] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research (JAIR)*, 2013.
- [60] Oleg Klimov. Openai gym: Rocket trajectory optimization is a classic topic in optimal control. <https://github.com/openai/gym>, 2019.
- [61] Antonin Raffin. RL baselines zoo. <https://github.com/araffin/rl-baselines-zoo>, 2018.
- [62] Ashley Hill, Antonin Raffin, Maximilian Ernestus, Adam Gleave, Anssi Kanervisto, Rene Traore, Prafulla Dhariwal, Christopher Hesse, Oleg Klimov, Alex Nichol, Matthias Plappert, Alec Radford, John Schulman, Szymon Sidor, and Yuhuai Wu. Stable baselines. <https://github.com/hill-a/stable-baselines>, 2018.
- [63] Alistair EW Johnson, Tom J Pollard, Lu Shen, H Lehman Li-wei, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, and Roger G Mark. Mimic-iii, a freely accessible critical care database. *Nature Scientific data*, 2016.
- [64] Donghun Lee, Srivatsan Srinivasan, and Finale Doshi-Velez. Batch apprenticeship learning. <https://github.com/dtak/batch-apprenticeship-learning>, 2019.
- [65] Ilya Kostrikov, Ofir Nachum, and Jonathan Tompson. Imitation learning via off-policy distribution matching. https://github.com/google-research/google-research/tree/master/value_dice, 2020.
- [66] Will Grathwohl, Kuan-Chieh Wang, Jörn-Henrik Jacobsen, David Duvenaud, Mohammad Norouzi, and Kevin Swersky. Jem - joint energy models. <https://github.com/wgrathwohl/JEM>, 2020.
- [67] Fredrik K Gustafsson, Martin Danelljan, Radu Timofte, and Thomas B Schön. How to train your energy-based model for regression. *arXiv preprint*, 2020.
- [68] Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. *Deep Learning*. MIT Press Cambridge, 2016.
- [69] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint*, 2017.
- [70] Sungjoon Choi, Kyungjae Lee, Andy Park, and Songhwai Oh. Density matching reward learning. *arXiv preprint*, 2016.
- [71] Fangchen Liu, Zhan Ling, Tongzhou Mu, and Hao Su. State alignment-based imitation learning. *International Conference on Learning Representations (ICLR)*, 2020.
- [72] Ruohan Wang, Carlo Ciliberto, Pierluigi Amadori, and Yiannis Demiris. Random expert distillation: Imitation learning via expert policy support estimation. *International Conference on Machine Learning (ICML)*, 2019.
- [73] Kianté Brantley, Wen Sun, and Mikael Henaff. Disagreement-regularized imitation learning. *International Conference on Learning Representations (ICLR)*, 2020.
- [74] Robert Dadashi, Leonard Hussenot, Matthieu Geist, and Olivier Pietquin. Primal wasserstein imitation learning. *arXiv preprint*, 2020.
- [75] Minghuan Liu, Tairan He, Minkai Xu, and Weinan Zhang. Energy-based imitation learning. *arXiv preprint*, 2020.
- [76] Matteo Pirotta and Marcello Restelli. Inverse reinforcement learning through policy gradient minimization. *AAAI Conference on Artificial Intelligence (AAAI)*, 2016.
- [77] Davide Tateo, Matteo Pirotta, Marcello Restelli, and Andrea Bonarini. Gradient-based minimization for multi-expert inverse reinforcement learning. *IEEE Symposium Series on Computational Intelligence (SSCI)*, 2017.
- [78] Alberto Maria Metelli, Matteo Pirotta, and Marcello Restelli. Compatible reward inverse reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

A Proofs of Propositions

Lemma 4 Let $\theta \in \Theta$ be some parameter, consider a random variable $s \in \mathcal{S}$, and fix $f : \mathcal{S} \times \Theta \rightarrow \mathbb{R}$, where $f(s, \theta)$ is continuously differentiable with respect to θ and integrable for all θ . Assume for some random variable X with finite mean that $|\frac{\partial}{\partial \theta} f(s, \theta)| \leq X$ holds almost surely for all θ . Then:

$$\frac{\partial}{\partial \theta} \mathbb{E}[f(s, \theta)] = \mathbb{E}[\frac{\partial}{\partial \theta} f(s, \theta)] \quad (17)$$

Proof. $\frac{\partial}{\partial \theta} \mathbb{E}[f(s, \theta)] = \lim_{\delta \rightarrow 0} \frac{1}{\delta} (\mathbb{E}[f(s, \theta + \delta)] - \mathbb{E}[f(s, \theta)]) = \lim_{\delta \rightarrow 0} \mathbb{E}[\frac{1}{\delta} (f(s, \theta + \delta) - f(s, \theta))]$
 $= \lim_{\delta \rightarrow 0} \mathbb{E}[\frac{\partial}{\partial \theta} f(s, \tau(\delta))] = \mathbb{E}[\lim_{\delta \rightarrow 0} \frac{\partial}{\partial \theta} f(s, \tau(\delta))] = \mathbb{E}[\frac{\partial}{\partial \theta} f(s, \theta)]$, where for the third equality the mean value theorem guarantees the existence of $\tau(\delta) \in (\theta, \theta + \delta)$, and the fourth equality uses the dominated convergence theorem where $|\frac{\partial}{\partial \theta} f(s, \tau(\delta))| \leq X$ by assumption. Note that generalizing to the multivariate case (i.e. gradients) simply requires that the bound be on $\max_i |\frac{\partial}{\partial \theta_i} f(s, \theta)|$ for elements i of θ . Note that most machine learning models (and energy-based models) meet/assume these regularity conditions or similar variants; see e.g. discussion presented in Section 18.1 in [68].

Proposition 1 (Surrogate Objective) Define the “occupancy” loss $\mathcal{L}_\rho$ as the difference in energy:

$$\mathcal{L}_\rho(\theta) \doteq \mathbb{E}_{s \sim \rho_D} E_\theta(s) - \mathbb{E}_{s \sim \rho_\theta} E_\theta(s) \quad (11)$$

Then $\nabla_\theta \mathcal{L}_\rho(\theta) = -\mathbb{E}_{s \sim \rho_D} \nabla_\theta \log \rho_\theta(s)$. In other words, differentiating this recovers the first term in Equation 9. Therefore if we define a standard “policy” loss $\mathcal{L}_\pi(\theta) \doteq -\mathbb{E}_{s, a \sim \rho_D} \log \pi_\theta(a|s)$, then:

$$\mathcal{L}_{\text{sur}}(\theta) \doteq \mathcal{L}_\rho(\theta) + \mathcal{L}_\pi(\theta) \quad (12)$$

yields a surrogate objective that can be optimized, instead of the original $\mathcal{L}$. Note that by relying on the offline energy-based model, we now have access to the gradients of the terms in the expectations.

Proof. For each s , first write the state occupancy measure as $\rho_\theta(s) = e^{-E_\theta(s)} / \int_{\mathcal{S}} e^{-E_\theta(s)} ds$, so:

$$-\log \rho_\theta(s) = E_\theta(s) + \log \int_{\mathcal{S}} e^{-E_\theta(s)} ds \quad (18)$$

with gradients given by:

$$\begin{aligned} -\nabla_\theta \log \rho_\theta(s) &= \nabla_\theta E_\theta(s) + \nabla_\theta \log \int_{\mathcal{S}} e^{-E_\theta(s)} ds \\ &= \nabla_\theta E_\theta(s) - \frac{\int_{\mathcal{S}} \nabla_\theta E_\theta(s) e^{-E_\theta(s)} ds}{\int_{\mathcal{S}} e^{-E_\theta(s)} ds} \\ &= \nabla_\theta E_\theta(s) - \mathbb{E}_{s \sim \rho_\theta} \nabla_\theta E_\theta(s) \end{aligned} \quad (19)$$

Then taking expectations over ρ_D and substituting in the energy term per Equation 10, we have that:

$$\begin{aligned} -\nabla_\theta \mathbb{E}_{s \sim \rho_D} \log \rho_\theta(s) &= \mathbb{E}_{s \sim \rho_D} [\nabla_\theta E_\theta(s) - \mathbb{E}_{s \sim \rho_\theta} \nabla_\theta E_\theta(s)] \\ &= \mathbb{E}_{s \sim \rho_D} \nabla_\theta E_\theta(s) - \mathbb{E}_{s \sim \rho_\theta} \nabla_\theta E_\theta(s) \\ &= \mathbb{E}_{s \sim \rho_\theta} \nabla_\theta (\log \sum_a e^{f_\theta(s)[a]}) - \mathbb{E}_{s \sim \rho_D} \nabla_\theta (\log \sum_a e^{f_\theta(s)[a]}) \\ &= \nabla_\theta (\mathbb{E}_{s \sim \rho_\theta} \log \sum_a e^{f_\theta(s)[a]} - \mathbb{E}_{s \sim \rho_D} \log \sum_a e^{f_\theta(s)[a]}) \\ &= \nabla_\theta \mathcal{L}_\rho(\theta) \end{aligned} \quad (20)$$

where the fourth equality uses Lemma 4. Hence we can define $\mathcal{L}_\rho(\theta) \doteq \mathbb{E}_{s \sim \rho_D} E_\theta(s) - \mathbb{E}_{s \sim \rho_\theta} E_\theta(s)$ in lieu of the first term in Equation 8. However, note that the (gradient-based) implementation of Algorithm 1 works even without first obtaining an expression for $\mathcal{L}_\rho(\theta)$ per se, and is correct due to a simpler reason: The batched (empirical loss) $\nabla_\theta \mathcal{L}_\rho$ portion of the update (Line 9) is directly analogous to the gradient update in standard contrastive divergence; see e.g. Section 18.2 in [68]. $\square$

Propositions 2–3 first require an additional lemma that allows moving freely between the space of (soft) Q -functions and reward functions. Recall the (soft) Bellman operator $\mathbb{B}_R^* : \mathbb{R}^{\mathcal{S} \times \mathcal{A}} \rightarrow \mathbb{R}^{\mathcal{S} \times \mathcal{A}}$:

$$(\mathbb{B}_R^* Q)(s, a) = R(s, a) + \gamma \mathbb{E}_T[\text{softmax}_{a'} Q(s', a') | s, a] \quad (21)$$

where $\text{softmax}_a Q(s, a) \doteq \log \sum_a e^{Q(s, a)}$. We know that $\mathbb{B}_R^*$ is contractive with Q_R^* its unique fixed point [10, 11]. Now, let us define the (soft) inverse Bellman operator $\mathbb{J}^* : \mathbb{R}^{\mathcal{S} \times \mathcal{A}} \rightarrow \mathbb{R}^{\mathcal{S} \times \mathcal{A}}$ such that:

$$(\mathbb{J}^* Q)(s, a) = Q(s, a) - \gamma \mathbb{E}_T[\text{softmax}_a Q(s', a') | s, a] \quad (22)$$

Lemma 5 The operator $\mathbb{J}^*$ is *bijective*: $Q = Q_R^* \Leftrightarrow \mathbb{J}^*Q = R$, hence we can write $(\mathbb{J}^*)^{-1}R = Q_R^*$. This is the “soft” version of an analogous statement made for “hard” optimality first shown in [32].

Proof. By the uniqueness of the fixed point of $\mathbb{B}_R^*$, we have that $R = \mathbb{J}^*Q \Leftrightarrow \mathbb{B}_R^*Q = Q \Leftrightarrow Q = Q_R^*$. Therefore the inverse image of every singleton $R \in \mathbb{R}^{\mathcal{S} \times \mathcal{A}}$ must exist, and is uniquely equal to Q_R^* . This argument is the direct counterpart to Theorem 2 in [32]—which uses argmax instead of softmax .

Proposition 2 (Classical Objective) Consider the classical IL objective in Equation 1, with policies parameterized as Equation 6. Choosing $\mathcal{L}$ to be the (forward) KL divergence yields the following:

$$\text{argmax}_R (\mathbb{E}_{s \sim \rho_R^*} \mathbb{E}_{a \sim \pi_D(\cdot|s)} Q_R^*(s, a) - \mathbb{E}_{s \sim \rho_R^*} V_R^*(s)) \quad (14)$$

where $Q_R^* : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the (soft) Q -function given by $Q_R^*(s, a) = R(s, a) + \gamma \mathbb{E}_T[V_R^*(s')|s, a]$, $V^*(s) \in \mathbb{R}^{\mathcal{S}}$ is the (soft) value function $V_R^*(s) = \log \sum_a e^{Q_R^*(s, a)}$, and ρ_R^* is the occupancy for π_R^* .

Proof. From Equations 1 and 6, choosing $\mathcal{L}$ to be the forward KL divergence yields the following:

$$\text{argmax}_\theta (\mathbb{E}_{s \sim \rho_\theta} \mathbb{E}_{a \sim \pi_D(\cdot|s)} f_\theta(s)[a] - \mathbb{E}_{s \sim \rho_\theta} \log \sum_a e^{f_\theta(s)[a]}) \quad (23)$$

Now, observe that we are free to identify the logits $f_\theta(s)[a] \in \mathbb{R}^{\mathcal{S} \times \mathcal{A}}$ with a (soft) Q -function. Specifically, define $Q(s, a) \doteq f_\theta(s)[a]$ for all $s, a \in \mathcal{S} \times \mathcal{A}$. Then by Lemma 5 we know there exists a unique $R \in \mathbb{R}^{\mathcal{S} \times \mathcal{A}}$ that $\mathbb{J}^*$ takes Q to. Hence $f_\theta(s)[a] = Q_R^*(s, a)$ for some R , and we can write:

$$\text{argmax}_R (\mathbb{E}_{s \sim \rho_R^*} \mathbb{E}_{a \sim \pi_D(\cdot|s)} Q_R^*(s, a) - \mathbb{E}_{s \sim \rho_R^*} \log \sum_a e^{Q_R^*(s, a)}) \quad (24)$$

where $\pi_R^*(a|s) = e^{Q_R^*(s, a) - V_R^*(s)}$. Then Proposition 2 follows, since $V_R^*(s) = \log \sum_a e^{Q_R^*(s, a)}$. $\square$

Proposition 3 (From BC to EDM) The behavioral cloning objective is equivalently the following, where—compared to Equation 14—expectations over states are now taken w.r.t. ρ_D instead of ρ_R^* :

$$\text{argmax}_R (\mathbb{E}_{s \sim \rho_D} \mathbb{E}_{a \sim \pi_D(\cdot|s)} Q_R^*(s, a) - \mathbb{E}_{s \sim \rho_D} V_R^*(s)) \quad (15)$$

In contrast, by augmenting the (behavioral cloning) “policy” loss $\mathcal{L}_\pi$ with the “occupancy” loss $\mathcal{L}_\rho$, what the EDM surrogate objective achieves is to replace one of the expectations with the learned ρ_θ :

$$\text{argmax}_R (\mathbb{E}_{s \sim \rho_D} \mathbb{E}_{a \sim \pi_D(\cdot|s)} Q_R^*(s, a) - \mathbb{E}_{s \sim \rho_\theta} V_R^*(s)) \quad (16)$$

Proof. By definition of behavioral cloning, the only difference is that the expectation in Equation 1 is taken over ρ_D ; then the same argument for Proposition 2 applies. As for EDM, from Equation 12:

$$\begin{aligned} \mathcal{L}_{\text{sur}}(\theta) &= \mathcal{L}_\rho(\theta) + \mathcal{L}_\pi(\theta) \\ &= \mathbb{E}_{s \sim \rho_D} E_\theta(s) - \mathbb{E}_{s \sim \rho_\theta} E_\theta(s) - \mathbb{E}_{s, a \sim \rho_D} \log \pi_\theta(a|s) \\ &= \mathbb{E}_{s \sim \rho_\theta} \log \sum_a e^{f_\theta(s)[a]} - \mathbb{E}_{s \sim \rho_D} \log \sum_a e^{f_\theta(s)[a]} \\ &\quad + \mathbb{E}_{s, a \sim \rho_D} \log \sum_a e^{f_\theta(s)[a]} - \mathbb{E}_{s, a \sim \rho_D} f_\theta(s)[a] \\ &= \mathbb{E}_{s \sim \rho_\theta} \log \sum_a e^{f_\theta(s)[a]} - \mathbb{E}_{s, a \sim \rho_D} f_\theta(s)[a] \end{aligned} \quad (25)$$

Therefore minimizing $\mathcal{L}_{\text{sur}}(\theta)$ is equivalent to:

$$\text{argmax}_\theta (\mathbb{E}_{s \sim \rho_D} \mathbb{E}_{a \sim \pi_D(\cdot|s)} f_\theta(s)[a] - \mathbb{E}_{s \sim \rho_\theta} \log \sum_a e^{f_\theta(s)[a]}) \quad (26)$$

From this point onwards, the same strategy for Proposition 2 again applies, completing the proof. $\square$

B Experiment Details

Gym Environments Environments used for experiments are from OpenAI gym [56]. Table 3 shows environment names and version numbers, dimensions of each observation space, and cardinalities of each action space. Each environment is associated with a true reward function (unknown to all imitation algorithms). In each case, the “expert” demonstrator is obtained using a pre-trained and hyperparameter-optimized agent from the RL Baselines Zoo [61] in Stable OpenAI Baselines [62]; for all environments, demonstration datasets $\mathcal{D}$ are generated using the PPO2 agent [69] trained on the true reward function, with the exception of CartPole, for which we use the DQN agent (which we find performs better than PPO2). Performance of demonstrator and random policies are shown:

<i>Environments</i>	Observation Space	Action Space	Demonstrator	Random Perf.	Demonstrator Perf.
CartPole-v1	Continuous (4)	Discrete (2)	DQN Agent	19.12 ± 1.76	500.00 ± 0.00
Acrobot-v1	Continuous (6)	Discrete (3)	PPO2 Agent	-439.92 ± 13.14	-87.32 ± 12.02
LunarLander-v2	Continuous (8)	Discrete (4)	PPO2 Agent	-452.22 ± 61.24	271.71 ± 17.88
BeamRider-v4	Cont. (210×160×3)	Discrete (9)	PPO2 Agent	954.84 ± 214.85	1623.80 ± 482.27
MIMIC-III-2a	Continuous (56)	Discrete (2)	Human Agent	-	-
MIMIC-III-4a	Continuous (56)	Discrete (4)	Human Agent	-	-

Table 3: *Details of Environments.* Demonstrator and random performances are computed using 1,000 episodes.

Healthcare Environments MIMIC-III is a real-world medical dataset consisting of patients treated in intensive care units from the Medical Information Mart for Intensive Care [63], which records physiological data streams for over 22,000 patients. We extract the records for ICU patients administered with antibiotic treatment and/or mechanical ventilation (5,833 in total). For each patient, we define the observation space to be the 28 most frequently measured patient covariates from the past two days, including vital signs (e.g. temperature, heart rate, blood pressure, oxygen saturation, respiratory rate, etc.) and lab tests (e.g. white blood cell count, glucose levels, etc.), aggregated on a daily basis during their ICU stay. Each patient trajectory has up to 20 time steps. In this environment, the action space consists of the possible treatment choices administered by the doctor every day over the course of the patient’s ICU stay, and the “expert” demonstrations are simply the trajectories of states and actions recorded in the dataset. We consider two versions of MIMIC-III; one with 2 actions: with ventilator support, or no treatment (MIMIC-III-2a), and another with 4 actions: with ventilator support, antibiotics treatment, ventilator support plus antibiotics, or no treatment (MIMIC-III-4a).

Detailed Results Exact experiment results are shown in Table 4. For each combination of gym environment, imitation algorithm, and dataset size, we follow convention for randomization in our experiment setup by rolling out multiple trajectories (n_{traj}) per trained policy, seeding the experiment multiple times with different expert demonstrations (n_{demo}), and training multiple such policies from different random initializations (n_{init}); see e.g. [12]. Here we set $n_{\text{traj}}=300$, $n_{\text{demo}}=10$, and $n_{\text{init}}=5$. Table 4 shows the means of performance metrics, as well as their standard errors; for ease of comparison, all numbers for gym environments are scaled (according to the performance of demonstrator and random policies given in Table 3) such that the demonstrator attains a return of 1 and the random policy attains a return of 0. For the real-world healthcare environments, we have no access to the ground-truth reward function, and we cannot perform live policy rollouts. We therefore assess imitation performance according to action-matching on held-out test trajectories; see e.g. [64]. In each of $n_{\text{demo}}=10$ folds, we use an 80%-20% train-test split (i.e. 4,666 patients for training, and 1,167 held out for testing). In each instance, we report accuracy of action selection (ACC), area under the receiving operator characteristic curve (AUC), and area under the precision-recall curve (APR).

Implementations Wherever possible, policies trained by all imitation algorithms share the same policy network architecture: two hidden (fully connected) layers of 64 units each, followed by ELU activations, or—for Atari—a convolutional neural network with 3 (convolutional) layers of 32-64-64 filters, followed by a fully connected layer with 64 units, with all layers followed by ReLU activations. For all environments, we use the Adam optimizer with batch size 64, 10k iterations, and learning rate $1e-3$. Except explicitly standardizing policy networks across imitation algorithms, all comparators are implemented via the original publicly available source code. Where applicable, we use the optimal hyperparameters in the original implementations. The source code for EDM is found at <https://bitbucket.org/mvdschaar/mlforhealthlabpub> & <https://github.com/danieljarrett/EDM>.

Hyperparameters for EDM Algorithm 1 is implemented using the source code for joint EBM [47] publicly available at <https://github.com/wgrathwohl/JEM>. Instead of Wide-Resnet, for Acrobot, Cartpole, LunarLander, MIMIC-III-2a, and MIMIC-III-4a we use the fully-connected policy network above, and for BeamRider the convolutional neural network above. Specific to EDM are the joint EBM training hyperparameters, which we inherit from [47, 66]: noise coefficient $\sigma=0.01$, buffer size $\kappa=10000$, length $\iota=20$, and reinitialization $\delta=0.05$. We find that these default settings work well with SGLD step size $\alpha=0.01$; for further EBM training-related discussions, we refer to [47, 48].

Hyperparameters for VDICE We take the original source code of [43], which is publicly available at https://github.com/google-research/google-research/tree/master/value_dice. In order to adapt the model to work with discrete action spaces, we use a Gumbel-softmax parameterization for the last layer of the actor network. For Acrobot, Cartpole, LunarLander, MIMIC-III-2a, and MIMIC-III-4a both the actor architecture and the discriminator architecture has two hidden (fully

		BC	RCAL	DSFN	VDICE	EDM
	<i>Demos</i>	<i>Average Returns</i>				
Acrobot-v1	1	0.796 $\pm$ 0.078	0.422 $\pm$ 0.082	0.062 $\pm$ 0.141	0.857 $\pm$ 0.045	0.896 $\pm$ 0.064
	3	0.976 $\pm$ 0.028	0.832 $\pm$ 0.066	0.227 $\pm$ 0.128	0.947 $\pm$ 0.033	0.998 $\pm$ 0.026
	7	0.981 $\pm$ 0.028	0.975 $\pm$ 0.034	0.489 $\pm$ 0.075	0.953 $\pm$ 0.036	0.999 $\pm$ 0.026
	10	0.986 $\pm$ 0.029	0.990 $\pm$ 0.030	0.601 $\pm$ 0.076	0.967 $\pm$ 0.032	0.999 $\pm$ 0.025
	15	0.994 $\pm$ 0.028	0.997 $\pm$ 0.028	0.825 $\pm$ 0.050	0.976 $\pm$ 0.031	1.000 $\pm$ 0.026
CartPole-v1	1	0.321 $\pm$ 0.026	0.233 $\pm$ 0.036	0.317 $\pm$ 0.013	0.324 $\pm$ 0.018	0.428 $\pm$ 0.019
	3	0.607 $\pm$ 0.048	0.586 $\pm$ 0.043	0.373 $\pm$ 0.073	0.738 $\pm$ 0.028	0.900 $\pm$ 0.029
	7	0.819 $\pm$ 0.041	0.894 $\pm$ 0.027	0.523 $\pm$ 0.081	0.867 $\pm$ 0.022	0.982 $\pm$ 0.011
	10	0.932 $\pm$ 0.026	0.991 $\pm$ 0.007	0.458 $\pm$ 0.047	0.967 $\pm$ 0.013	1.000 $\pm$ 0.001
	15	0.997 $\pm$ 0.003	0.998 $\pm$ 0.001	0.653 $\pm$ 0.074	0.995 $\pm$ 0.004	0.998 $\pm$ 0.002
LunarLander-v2	1	0.575 $\pm$ 0.071	0.540 $\pm$ 0.090	0.229 $\pm$ 0.104	0.255 $\pm$ 0.071	0.633 $\pm$ 0.081
	3	0.869 $\pm$ 0.055	0.875 $\pm$ 0.055	0.698 $\pm$ 0.050	0.385 $\pm$ 0.063	0.889 $\pm$ 0.069
	7	0.938 $\pm$ 0.035	0.914 $\pm$ 0.057	0.776 $\pm$ 0.053	0.411 $\pm$ 0.063	0.956 $\pm$ 0.044
	10	0.961 $\pm$ 0.035	0.952 $\pm$ 0.047	0.887 $\pm$ 0.042	0.418 $\pm$ 0.059	0.966 $\pm$ 0.040
	15	0.968 $\pm$ 0.028	0.970 $\pm$ 0.028	0.913 $\pm$ 0.032	0.417 $\pm$ 0.054	0.970 $\pm$ 0.033
BeamRider-v4	1	0.124 $\pm$ 0.168	0.304 $\pm$ 0.195	0.000 $\pm$ 0.340	0.180 $\pm$ 0.159	0.486 $\pm$ 0.235
	3	0.147 $\pm$ 0.179	0.461 $\pm$ 0.227	0.008 $\pm$ 0.376	0.332 $\pm$ 0.205	0.790 $\pm$ 0.277
	7	0.270 $\pm$ 0.179	0.547 $\pm$ 0.239	0.140 $\pm$ 0.463	0.312 $\pm$ 0.175	0.839 $\pm$ 0.289
	10	0.308 $\pm$ 0.168	0.668 $\pm$ 0.279	0.153 $\pm$ 0.329	0.534 $\pm$ 0.227	0.925 $\pm$ 0.278
	15	0.401 $\pm$ 0.169	0.721 $\pm$ 0.202	0.082 $\pm$ 0.301	0.513 $\pm$ 0.211	0.991 $\pm$ 0.272
	<i>Metrics</i>	<i>Action-Matching</i>				
MIMIC-III-2a	ACC	0.861 $\pm$ 0.013	0.872 $\pm$ 0.007	0.865 $\pm$ 0.007	0.875 $\pm$ 0.004	0.891 $\pm$ 0.004
	AUC	0.914 $\pm$ 0.003	0.911 $\pm$ 0.007	0.906 $\pm$ 0.003	0.915 $\pm$ 0.001	0.922 $\pm$ 0.004
	APR	0.902 $\pm$ 0.005	0.898 $\pm$ 0.006	0.885 $\pm$ 0.001	0.904 $\pm$ 0.002	0.912 $\pm$ 0.005
MIMIC-III-4a	ACC	0.696 $\pm$ 0.006	0.701 $\pm$ 0.007	0.682 $\pm$ 0.005	0.707 $\pm$ 0.005	0.720 $\pm$ 0.007
	AUC	0.859 $\pm$ 0.003	0.864 $\pm$ 0.003	0.857 $\pm$ 0.002	0.864 $\pm$ 0.002	0.873 $\pm$ 0.002
	APR	0.659 $\pm$ 0.007	0.667 $\pm$ 0.006	0.665 $\pm$ 0.003	0.673 $\pm$ 0.003	0.681 $\pm$ 0.008

Table 4: Detailed Results for Gym and Healthcare Environments. Bold numbering indicates best performance.

connected) layers of 64 units each with ReLU activation, and—for Atari—the actor and discriminator are replaced with convolutional neural networks with 3 (convolutional) layers of 32-64-64 filters followed by a fully connected layer with 64 units, with all layers followed by ReLU activations. Per the original design, the output is concatenated with the action; this is then passed through 2 additional hidden layers with 64 units each. In addition, to enable strictly batch learning, we set the “replay regularization” coefficient to zero. Furthermore, the actor network is regularized with an “orthogonal regularization” coefficient of 1e-4, actor learning rate of 1e-5, and discriminator learning rate of 1e-3.

Hyperparameters for DSFN We take the original source code of [43], which is publicly available at <https://github.com/dtak/batch-apprenticeship-learning>. Per [37], for Acrobot, Cartpole, LunarLander, MIMIC-III-2a, and MIMIC-III-4a we use a “warm-start” policy network with two shared layers of 128 and 64 dimensions and tanh activation. The hidden layer of size 64 is used as the feature map in the IRL algorithm. Each multitask head in the warm-start policy network has a hidden layer with 128 units and tanh activation. The DQN network (i.e. for learning the optimal policy given a set of reward weights) has 2 hidden (fully-connected) layers with 64 units each, and likewise the DSFN network for estimating feature expectations also has 2 hidden (fully-connected) layers with 64 units. For BeamRider, the first hidden layer in the warm-start policy network is replaced by a convolutional neural network with 3 layers of 32-64-64 filters, and the DQN and DSFN networks are also replaced by the convolutional neural network above. For all environments, the warm-start policy network is trained for 50k steps with the Adam optimizer, learning rate 3e-4, and batch size 64. The DQN network is trained for 30k steps with learning rate 3e-4 and batch size 64 (Adam). Finally, the DSFN network is trained for 50,000 iterations with the learning rate 3e-4 and batch size 32 (Adam).

Hyperparameters for RCAL This augments the policy loss with an additional sparsity-based loss on the implied rewards $\hat{R}(s, a) \doteq f_\theta(s)[a] - \gamma \text{softmax}_{a'} f_\theta(s')[a']$ obtained by inverting the Bellman equation [9, 32]. For Acrobot, Cartpole, LunarLander, MIMIC-III-2a, and MIMIC-III-4a we use the fully-connected policy network described above, and for BeamRider the convolutional neural network above. Specific to RCAL is its sparsity-based regularization coefficient, which is set at 1e-2.

Hyperparameters for BC The only difference between BC and EDM is the presence of $\mathcal{L}_\rho$, which we remove for our implementation of BC. (Unlike e.g. [32], we do not consider more primitive methods such as linear classifiers/trees to serve as BC, which would not make for a fair comparison/

ablation). For Acrobot, Cartpole, LunarLander, MIMIC-III-2a, and MIMIC-III-4a we use the fully-connected policy network above, and for BeamRider the convolutional neural network above.

Semi-Supervised Learning While this is beyond the scope of this work, we briefly note that—by analogy to joint energy-based modeling in general [47]—the EDM algorithm can additionally benefit from semi-supervised learning. Specifically, consider a data-scarce setting where we only have access to limited state-action pairs from the demonstrator—but may have access to additional state-only data. Broadly, this situation arises whenever states are more conveniently observed than actions are. For CartPole, Figure 3 shows the results of the original EDM trained on one demonstrator trajectory’s worth of state-action pairs, but with access to additional state-only data (**EDM-1t+**) shown in the x -axis as multiples of the original amount of state-action data. For comparison, we also reference the performance of EDM without such additional state-only data (EDM-1t), as well as the performance of its closest competitor (VDICE-1t), both trained on one trajectory’s worth of state-action pairs alone. Notably, observe that (purely by dint of state-only distribution matching) EDM-1t+ manages to extract a sizable gain in performance as the amount of state-only data available increases up to seven-fold. While this improvement is—as expected—less than that conferred by simply adding more state-action trajectories (cf. EDM-3t, which is trained on 3 trajectories’ worth of state-action pairs), simply adding state-only data manages to provide as much of a performance boost as the original difference between EDM and VDICE (trained on one trajectory’s worth of state-action pairs).

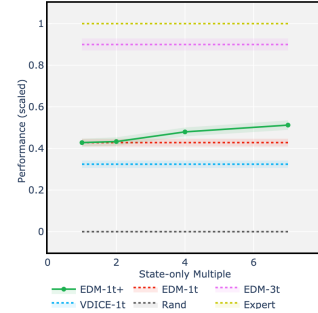


Figure 3: *Semi-Supervised Learning*.

C Further Related Work

Throughout this work, we discussed the goal of imitation learning [1–3] in the strictly batch setting, behavioral cloning [4–7] and its relatives [9,30–33], and relationships with the apprenticeship learning family of techniques, including classic (online) inverse reinforcement learning [13–20], (online) adversarial imitation learning [12,22–29], as well as their respective off-policy relatives [34–43,54]. Table 1 summarizes the major aspects of these works as pertinent to our discussion and development.

Further to these works, we also note that another line of research on (online) imitation learning seeks to incentivize the imitating policy to remain within the distribution/support of states encountered in the expert demonstrations [50,55,70–75]. For instance, this is approached through random expert distillation [72], through ensembles of agents [73], or the simple and elegant approach of assigning a unit reward to all demonstrated actions that occur in demonstrated states, and zero otherwise [55]. In general, these methods follow a “two-step” formula, where in the first step some notion of a surrogate reward function is derived/defined, and in the second step this reward function is optimized by way of environment interactions (and as such, they are inherently online techniques). In the same vein, while [75] bears some superficial resemblance to our method by way of energy-based modeling, it is an inherently online technique that depends on training an agent against an explicitly estimated reward function: In the first step, their reward function is defined by modeling the negative energy of the (joint) state-action distribution. However, as with the aforementioned two-step approaches, this must then be followed by an online optimization of this reward function—and is therefore inoperable in our strictly batch setting. Moreover, not unlike in adversarial imitation learning, their KL-divergence minimization interpretation similarly requires the assumption that the optimal reward function is indeed attained—an issue our formulation does not encounter. In contrast, EDM works by decomposing the state-action distribution into an (explicit) policy term and an (implicit) state visitation distribution term, resulting in a single optimization that works in an entirely offline manner.

Finally, tangentially related to our work is a family of inverse reinforcement learning methods designed for reward learning in an offline, model-free setting [76–78]. However, they require access to the demonstrator’s policy itself to begin with, and their objective is rather in the inverse problem *per se*—that is, of explicitly recovering the underlying reward function in order to understand behavior.

References

- [1] Hoang M Le, Andrew Kang, Yisong Yue, and Peter Carr. Smooth imitation learning for online sequence prediction. *International Conference on Machine Learning (ICML)*, 2016.

- [2] Ahmed Hussein, Mohamed Medhat Gaber, Eyad Elyan, and Chrisina Jayne. Imitation learning: A survey of learning methods. *ACM Computing Surveys (CSUR)*, 2017.
- [3] Yisong Yue and Hoang M Le. Imitation learning (presentation). *International Conference on Machine Learning (ICML)*, 2018.
- [4] Dean A Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural computation (NC)*, 1991.
- [5] Michael Bain and Claude Sammut. A framework for behavioural cloning. *Machine Intelligence (MI)*, 1999.
- [6] Umar Syed and Robert E Schapire. A reduction from apprenticeship learning to classification. *Advances in neural information processing systems (NeurIPS)*, 2010.
- [7] Stéphane Ross and Drew Bagnell. Efficient reductions for imitation learning. *International conference on artificial intelligence and statistics (AISTATS)*, 2010.
- [8] Francisco S Melo and Manuel Lopes. Learning from demonstration using mdp induced metrics. *Joint European conference on machine learning and knowledge discovery in databases (ECML)*, 2010.
- [9] Bilal Piot, Matthieu Geist, and Olivier Pietquin. Boosted and reward-regularized classification for apprenticeship learning. *International conference on Autonomous agents and multi-agent systems (AAMAS)*, 2014.
- [10] Brian D Ziebart. Modeling purposeful adaptive behavior with the principle of maximum causal entropy. *Phd Dissertation, Carnegie Mellon University*, 2010.
- [11] Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. *International Conference on Machine Learning (ICML)*, 2017.
- [12] Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. *Advances in neural information processing systems (NeurIPS)*, 2016.
- [13] Andrew Y Ng, Stuart J Russell, et al. Algorithms for inverse reinforcement learning. *International conference on Machine learning (ICML)*, 2000.
- [14] Pieter Abbeel and Andrew Y Ng. Apprenticeship learning via inverse reinforcement learning. *International conference on Machine learning (ICML)*, 2004.
- [15] Gergely Neu and Csaba Szepesvári. Apprenticeship learning using irl and gradient methods. *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2007.
- [16] Deepak Ramachandran and Eyal Amir. Bayesian inverse reinforcement learning. *International Joint Conference on Artificial Intelligence (IJCAI)*, 2007.
- [17] Umar Syed and Robert E Schapire. A game-theoretic approach to apprenticeship learning. *Advances in neural information processing systems (NeurIPS)*, 2008.
- [18] Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, and Anind K Dey. Maximum entropy inverse reinforcement learning. *AAAI Conference on Artificial Intelligence (AAAI)*, 2008.
- [19] Monica Babes, Vukosi Marivate, and Michael L Littman. Apprenticeship learning about multiple intentions. *International conference on Machine learning (ICML)*, 2011.
- [20] Jaedeug Choi and Kee-Eung Kim. Map inference for bayesian inverse reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2011.
- [21] Daniel Jarrett and Mihaela van der Schaar. Inverse active sensing: Modeling and understanding timely decision-making. *International Conference on Machine Learning*, 2020.
- [22] Nir Baram, Oron Anschel, and Shie Mannor. Model-based adversarial imitation learning. *International Conference on Machine Learning (ICML)*, 2017.
- [23] Wonseok Jeon, Seokin Seo, and Kee-Eung Kim. A bayesian approach to generative adversarial imitation learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [24] Chelsea Finn, Paul Christiano, Pieter Abbeel, and Sergey Levine. A connection between generative adversarial networks, inverse reinforcement learning, and energy-based models. *NeurIPS 2016 Workshop on Adversarial Training*, 2016.
- [25] Justin Fu, Katie Luo, and Sergey Levine. Learning robust rewards with adversarial inverse reinforcement learning. *International Conference on Learning Representations (ICLR)*, 2018.
- [26] Ahmed H Qureshi, Byron Boots, and Michael C Yip. Adversarial imitation via variational inverse reinforcement learning. *International Conference on Learning Representations (ICLR)*, 2019.
- [27] Seyed Kamyar Seyed Ghasemipour, Richard Zemel, and Shixiang Gu. A divergence minimization perspective on imitation learning methods. *Conference on Robot Learning (CoRL)*, 2019.

- [28] Kee-Eung Kim and Hyun Soo Park. Imitation learning via kernel mean embedding. *AAAI Conference on Artificial Intelligence (AAAI)*, 2018.
- [29] Huang Xiao, Michael Herman, Joerg Wagner, Sebastian Ziesche, Jalal Etesami, and Thai Hong Linh. Wasserstein adversarial imitation learning. *arXiv preprint*, 2019.
- [30] Umar Syed and Robert E Schapire. Imitation learning with a value-based prior. *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2007.
- [31] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *International conference on artificial intelligence and statistics (AISTATS)*, 2011.
- [32] Bilal Piot, Matthieu Geist, and Olivier Pietquin. Bridging the gap between imitation learning and irl. *IEEE transactions on neural networks and learning systems*, 2017.
- [33] Alexandre Attia and Sharone Dayan. Global overview of imitation learning. *arXiv preprint*, 2018.
- [34] Edouard Klein, Matthieu Geist, and Olivier Pietquin. Batch, off-policy and model-free apprenticeship learning. *European Workshop on Reinforcement Learning (EWRL)*, 2011.
- [35] Takeshi Mori, Matthew Howard, and Sethu Vijayakumar. Model-free apprenticeship learning for transfer of human impedance behaviour. *IEEE-RAS International Conference on Humanoid Robots*, 2011.
- [36] Vinamra Jain, Prashant Doshi, and Bikramjit Banerjee. Model-free irl using maximum likelihood estimation. *AAAI Conference on Artificial Intelligence (AAAI)*, 2019.
- [37] Donghun Lee, Srivatsan Srinivasan, and Finale Doshi-Velez. Truly batch apprenticeship learning with deep successor features. *International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.
- [38] Aristide CY Tossou and Christos Dimitrakakis. Probabilistic inverse reinforcement learning in unknown environments. *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2013.
- [39] Michael Herman, Tobias Gindele, Jörg Wagner, Felix Schmitt, and Wolfram Burgard. Inverse reinforcement learning with simultaneous estimation of rewards and dynamics. *International conference on artificial intelligence and statistics (AISTATS)*, 2016.
- [40] Ajay Kumar Tanwani and Aude Billard. Inverse reinforcement learning for compliant manipulation in letter handwriting. *National Center of Competence in Robotics (NCCR)*, 2013.
- [41] Lionel Blondé and Alexandros Kalousis. Sample-efficient imitation learning via gans. *International conference on artificial intelligence and statistics (AISTATS)*, 2019.
- [42] Ilya Kostrikov, Kumar Krishna Agrawal, Debidatta Dwibedi, Sergey Levine, and Jonathan Tompson. Discriminator-actor-critic: Addressing sample inefficiency and reward bias in adversarial imitation. *International Conference on Learning Representations (ICLR)*, 2019.
- [43] Ilya Kostrikov, Ofir Nachum, and Jonathan Tompson. Imitation learning via off-policy distribution matching. *International Conference on Learning Representations (ICLR)*, 2020.
- [44] Eugene A Feinberg and Adam Shwartz. *Markov decision processes: methods and applications*. Springer Science & Business Media, 2012.
- [45] Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.
- [46] Yann LeCun, Sumit Chopra, Raia Hadsell, M Ranzato, and F Huang. A tutorial on energy-based learning. *Predicting structured data*, 2006.
- [47] Will Grathwohl, Kuan-Chieh Wang, Jörn-Henrik Jacobsen, David Duvenaud, Mohammad Norouzi, and Kevin Swersky. Your classifier is secretly an energy based model and you should treat it like one. *International Conference on Learning Representations (ICLR)*, 2020.
- [48] Yilun Du and Igor Mordatch. Implicit generation and generalization in energy-based models. *Advances in neural information processing systems (NeurIPS)*, 2019.
- [49] Jianwen Xie, Yang Lu, Song-Chun Zhu, and Yingnian Wu. A theory of generative convnet. *International Conference on Machine Learning (ICML)*, 2016.
- [50] Yannick Schroecker and Charles L Isbell. State aware imitation learning. *Advances in neural information processing systems (NeurIPS)*, 2017.
- [51] Max Welling and Yee W Teh. Bayesian learning via stochastic gradient langevin dynamics. *International Conference on Machine Learning (ICML)*, 2011.
- [52] Erik Nijkamp, Mitch Hill, Tian Han, Song-Chun Zhu, and Ying Nian Wu. On the anatomy of mcmc-based maximum likelihood learning of energy-based models. *AAAI Conference on Artificial Intelligence (AAAI)*, 2020.

- [53] Tijmen Tieleman. Training restricted boltzmann machines using approximations to the likelihood gradient. *International Conference on Machine Learning (ICML)*, 2008.
- [54] Edouard Klein, Matthieu Geist, Bilal Piot, and Olivier Pietquin. Irl through structured classification. *Advances in neural information processing systems (NeurIPS)*, 2012.
- [55] Siddharth Reddy, Anca D Dragan, and Sergey Levine. Sqil: Imitation learning via regularized behavioral cloning. *International Conference on Learning Representations (ICLR)*, 2020.
- [56] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *OpenAI*, 2016.
- [57] Andrew G Barto, Richard S Sutton, and Charles W Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE transactions on systems, man, and cybernetics*, 1983.
- [58] Alborz Geramifard, Christoph Dann, Robert H Klein, William Dabney, and Jonathan P How. Rlpy: a value-function-based reinforcement learning framework for education and research. *Journal of Machine Learning Research (JMLR)*, 2015.
- [59] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research (JAIR)*, 2013.
- [60] Oleg Klimov. Openai gym: Rocket trajectory optimization is a classic topic in optimal control. <https://github.com/openai/gym>, 2019.
- [61] Antonin Raffin. RL baselines zoo. <https://github.com/araffin/rl-baselines-zoo>, 2018.
- [62] Ashley Hill, Antonin Raffin, Maximilian Ernestus, Adam Gleave, Anssi Kanervisto, Rene Traore, Prafulla Dhariwal, Christopher Hesse, Oleg Klimov, Alex Nichol, Matthias Plappert, Alec Radford, John Schulman, Szymon Sidor, and Yuhuai Wu. Stable baselines. <https://github.com/hill-a/stable-baselines>, 2018.
- [63] Alistair EW Johnson, Tom J Pollard, Lu Shen, H Lehman Li-wei, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, and Roger G Mark. Mimic-iii, a freely accessible critical care database. *Nature Scientific data*, 2016.
- [64] Donghun Lee, Srivatsan Srinivasan, and Finale Doshi-Velez. Batch apprenticeship learning. <https://github.com/dtak/batch-apprenticeship-learning>, 2019.
- [65] Ilya Kostrikov, Ofir Nachum, and Jonathan Tompson. Imitation learning via off-policy distribution matching. https://github.com/google-research/google-research/tree/master/value_dice, 2020.
- [66] Will Grathwohl, Kuan-Chieh Wang, Jörn-Henrik Jacobsen, David Duvenaud, Mohammad Norouzi, and Kevin Swersky. Jem - joint energy models. <https://github.com/wgrathwohl/JEM>, 2020.
- [67] Fredrik K Gustafsson, Martin Danelljan, Radu Timofte, and Thomas B Schön. How to train your energy-based model for regression. *arXiv preprint*, 2020.
- [68] Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. *Deep Learning*. MIT Press Cambridge, 2016.
- [69] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint*, 2017.
- [70] Sungjoon Choi, Kyungjae Lee, Andy Park, and Songhwai Oh. Density matching reward learning. *arXiv preprint*, 2016.
- [71] Fangchen Liu, Zhan Ling, Tongzhou Mu, and Hao Su. State alignment-based imitation learning. *International Conference on Learning Representations (ICLR)*, 2020.
- [72] Ruohan Wang, Carlo Ciliberto, Pierluigi Amadori, and Yiannis Demiris. Random expert distillation: Imitation learning via expert policy support estimation. *International Conference on Machine Learning (ICML)*, 2019.
- [73] Kianté Brantley, Wen Sun, and Mikael Henaff. Disagreement-regularized imitation learning. *International Conference on Learning Representations (ICLR)*, 2020.
- [74] Robert Dadashi, Leonard Hussenot, Matthieu Geist, and Olivier Pietquin. Primal wasserstein imitation learning. *arXiv preprint*, 2020.
- [75] Minghuan Liu, Tairan He, Minkai Xu, and Weinan Zhang. Energy-based imitation learning. *arXiv preprint*, 2020.
- [76] Matteo Pirota and Marcello Restelli. Inverse reinforcement learning through policy gradient minimization. *AAAI Conference on Artificial Intelligence (AAAI)*, 2016.
- [77] Davide Tateo, Matteo Pirota, Marcello Restelli, and Andrea Bonarini. Gradient-based minimization for multi-expert inverse reinforcement learning. *IEEE Symposium Series on Computational Intelligence (SSCI)*, 2017.
- [78] Alberto Maria Metelli, Matteo Pirota, and Marcello Restelli. Compatible reward inverse reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.